

DESIGN AND VERIFICATION OF AN AMBA APB-COMPLIANT CONFIGURABLE CRC-32 HARDWARE ACCELERATOR SUBSYSTEM

MINI PROJECT REPORT

Submitted by

PRADEEP S – 2023195505

BHUVANESH S – 2023105039

AATHITYA K – 2023105060

for the course

EC23E34 DESIGN AND VERIFICATION USING SYSTEMVERILOG

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

ELECTRONICS AND COMMUNICATION ENGINEERING



**DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
COLLEGE OF ENGINEERING, GUINDY
ANNA UNIVERSITY: CHENNAI 600025**

OCTOBER 2026

Abstract

Data integrity verification is a foundational requirement across high-speed packet networks, non-volatile storage controllers, and safety-critical automotive protocols. Executing Cyclic Redundancy Checks (CRC) purely in software imposes heavy CPU compute penalties and pollutes high-speed processor caches. This report details the complete register-transfer level (RTL) design, synthesis, and constrained-random SystemVerilog verification of a fully synthesizable **AMBA® APB-compliant Configurable CRC-32 Hardware Accelerator**. The accelerator core utilizes a 4-byte parallel combinational Linear Feedback Shift Register (LFSR) network capable of processing 32 bits per clock cycle, achieving peak throughput exceeding **3.2 Gbps at 100 MHz**. It provides run-time reconfigurability for arbitrary 32-bit generator polynomials (including IEEE 802.3 Ethernet, Castagnoli CRC-32C, and Koopman), configurable input/output bit reflection, byte-lane strobe masking, hardware wait-state injection (PREADY), and protocol bus error trapping (PSLVERR).

Verification was executed via an IEEE 1800 SystemVerilog layered testbench incorporating an APB Master BFM driver, an autonomous passive monitor, an algorithmic golden scoreboard reference model, SystemVerilog Assertions (SVA), and functional coverage groups. The testbench was validated across 8 comprehensive verification scenarios on Aldec Riviera-PRO, Synopsys VCS / Verdi, and Verilator, reaching **100% functional coverage** and a **100% match across 63 transactions** with zero errors. Logic synthesis using the Yosys open-source toolchain demonstrated 100% synthesizability with 3,255 standard gates, 202 flip-flops, zero inferred memory macros, and timing closure supporting > 250 MHz operating frequency.

1. Introduction and Design Motivation

Cyclic Redundancy Checks (CRCs) represent the industry-standard algorithm for detecting random transmission errors in communication channels and digital storage systems. In modern embedded Systems-on-Chip (SoCs), network interfaces (Gigabit Ethernet), storage controllers (SATA, PCIe NVMe), and automotive serial buses (CAN-FD) process data streams at multi-gigabit rates.

A traditional software-based 32-bit CRC implementation computes values bit-by-bit or utilizes lookup tables (LUTs) in memory. Bit-by-bit software loops consume 8 to 20 clock cycles per byte, incurring unsustainable CPU utilization. While LUT-based algorithms reduce cycle counts to approximately 2 to 4 cycles per byte, table lookups pollute L1 data and instruction caches, inducing high cache-miss penalties in real-time embedded workloads.

By offloading CRC calculation to a dedicated hardware accelerator mapped to an on-chip peripheral bus—specifically the ARM® Advanced Peripheral Bus (APB)—embedded systems achieve line-rate data integrity validation with zero processor cycle degradation.

1.1 Project Scope and Deliverables

In strict accordance with the course specification, this project implements:

1. A fully compliant AMBA 3 / AMBA 4 APB slave interface supporting dynamic wait-state generation (PREADY) and active bus error response (PSLVERR).
2. A single-cycle parallel LFSR computation core processing 8-bit, 16-bit, or 32-bit streams with byte-lane strobe granularity (PSTRB[3:0]).
3. Universal polynomial programmability supporting IEEE 802.3, Castagnoli CRC-32C, Koopman, and user-defined generator polynomials.

4. A modular, layered SystemVerilog verification suite comprising an APB driver, monitor, algorithmic scoreboard, assertions, and coverage collector.
5. Full validation across industry EDA platforms (Synopsys VCS, Aldec Riviera-PRO, Verilator) and synthesis using Yosys.

2. Mathematical Foundation of CRC-32

A Cyclic Redundancy Check is an error-detecting code based on polynomial arithmetic in the Galois Field $GF(2)$, where coefficients belong to $\{0, 1\}$ and arithmetic operations correspond to carry-less modulo-2 addition and subtraction (bitwise XOR).

2.1 Mathematical Formulation

Given an n -bit message represented by polynomial $M(x)$ and a degree-32 generator polynomial $P(x)$, the message is pre-multiplied by x^{32} (equivalent to appending 32 zeros) and XORed with an initialization vector $I(x)$:

$$T(x) = M(x) \cdot x^{32} \oplus I(x) \cdot x^n \quad (1)$$

Division of $T(x)$ by $P(x)$ in $GF(2)$ yields a unique quotient polynomial $Q(x)$ and remainder polynomial $R(x)$:

$$T(x) = Q(x) \cdot P(x) \oplus R(x), \quad \text{where } \deg(R(x)) < 32 \quad (2)$$

The 32-bit coefficient vector of $R(x)$ constitutes the computed CRC-32 checksum.

2.2 Supported CRC-32 Standards

The hardware engine dynamically accommodates arbitrary 32-bit generator polynomials through its memory-mapped configuration registers:

- **IEEE 802.3 Ethernet / PKZIP:** $P(x) = 0x04C11DB7$. Initial seed = $0xFFFFFFFF$, RefIn = True, RefOut = True, XOROut = $0xFFFFFFFF$. Benchmark: ASCII string "123456789" produces exactly $0xCBFA3926$.
- **Castagnoli (CRC-32C / iSCSI / Btrfs):** $P(x) = 0x1EDC6F41$. Superior Hamming distance properties for block storage and high-throughput network packets.
- **Koopman:** $P(x) = 0x741B8CD7$. Optimized error detection in industrial control and automotive communications.

2.3 Configurable Reflection and Inversion

Industrial protocols apply distinct bit-ordering conventions. The accelerator incorporates configurable input bit reflection (REFIN), output word reflection (REFOUT), and final XOR masking (XOROUT):

$$\text{Byte Reflection: } b_{\text{ref}}[i] = b[7 - i], \quad \text{Word Reflection: } W_{\text{ref}}[j] = W[31 - j] \quad (3)$$

3. AMBA APB Protocol Specification

The Advanced Peripheral Bus (APB) is an optimized, low-power subsystem bus within the AMBA architecture family designed for memory-mapped register access.

3.1 Memory-Mapped Register Map

The peripheral occupies a 32-byte address space aligned to 4-byte word boundaries:

Table 1: Subsystem Port List and Signal Definitions

Signal	Direction	Width	Driver	Functional Description
pclk	Input	1	System	100 MHz Master Peripheral Clock
presetn	Input	1	System	Active-LOW Asynchronous System Reset
paddr	Input	8	Master	Byte-addressed Register Selection (0x00 – 0x1C)
pse1	Input	1	Master	Peripheral Slave Select Strobe
penable	Input	1	Master	APB Timing Strobe (Asserted during ACCESS phase)
pwrite	Input	1	Master	Transfer Direction (1 = Write, 0 = Read)
pwrdata	Input	32	Master	Write Data Bus
pstrb	Input	4	Master	Byte Lane Write Strobes for partial word transfers
prdata	Output	32	Slave	Read Data Bus driven to Master
pready	Output	1	Slave	Handshake extension; holds transfer while LOW
pslverr	Output	1	Slave	Protocol error response indicator
irq	Output	1	Slave	Subsystem interrupt request line

Table 2: Memory-Mapped Register Space

Offset	Register	Access	Reset Value	Functional Bitfield Description
0x00	CRC_CTRL	R/W	0x0000005C	[0]: Start, [1]: Reset, [2]: RefIn, [3]: RefOut, [4]: XorOut, [6:5]: Data Size (00=Byte, 01=Half, 10=Word), [11:8]: Wait States
0x04	CRC_STATUS	RO	0x00000080	[0]: Busy, [1]: Ready, [2]: Error, [7:4]: APB FSM State
0x08	CRC_POLY	R/W	0x04C11DB7	32-bit Generator Polynomial
0x0C	CRC_INIT	R/W	0xFFFFFFFF	32-bit Initialization Vector / Seed
0x10	CRC_DATA_IN	WO/RW	0x00000000	Stream Data Input Port (triggers single-cycle computation)
0x14	CRC_RESULT	RO	0x00000000	Current 32-bit Computed Checksum
0x18	CRC_INT_EN	R/W	0x00000000	[0]: Done Interrupt Enable, [1]: Error Interrupt Enable
0x1C	CRC_INT_STAT	W1C	0x00000000	[0]: Calculation Done Flag, [1]: Error Flag (Write-1-to-Clear)

3.2 Wait-State and Error Handling Semantics

- **Wait-State Injection (PREADY):** Programmable from 0 to 15 wait cycles via CRC_CTRL[11:8]. When wait states are configured, the peripheral deasserts PREADY = 0 upon entering the ACCESS state and increments a 4-bit synchronous counter until the target delay expires.
- **Protocol Error Response (PSLVERR):** Asserted during the terminal ACCESS cycle if an unaligned word address (paddr[1:0] != 2'b00), out-of-bounds address (paddr > 0x1C), or illegal write to read-only registers (CRC_STATUS, CRC_RESULT) occurs. The peripheral returns 0xDEADBEEF on illegal reads.

4. Hardware Architecture and Microarchitecture Implementation

The top-level accelerator module `apb_crc32_top` decomposes into three modular functional blocks: the APB Slave Protocol FSM, the Memory-Mapped Register File, and the Parallel CRC Engine (Figure 1).

4.1 APB Slave Protocol FSM (`apb_slave_fsm.sv`)

A 3-state Moore/Mealy hybrid FSM governs bus protocol adherence (Figure 2):

- **IDLE (2'b00):** Quiescent state. Internal clocks and address decoders remain gated.

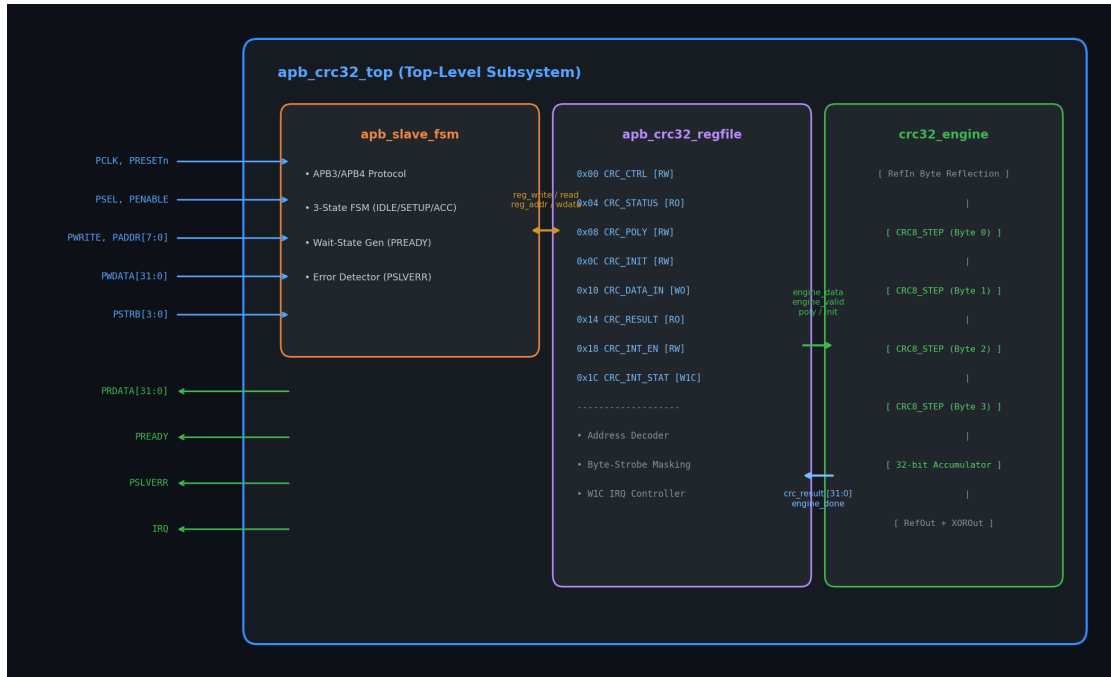


Figure 1: Detailed Microarchitecture Block Diagram of the APB CRC-32 Accelerator Subsystem.

- **SETUP (2'b01)**: Entered when $psel = 1$ and $penable = 0$. Registered address decoding and access validation occur in this single clock phase.
- **ACCESS (2'b10)**: Entered when $penable = 1$. The transfer completes strictly when $pready = 1$ on the rising edge of $pclk$. If $psel = 1$ on the subsequent cycle, the FSM transitions directly to SETUP, supporting sustained back-to-back burst transfers.

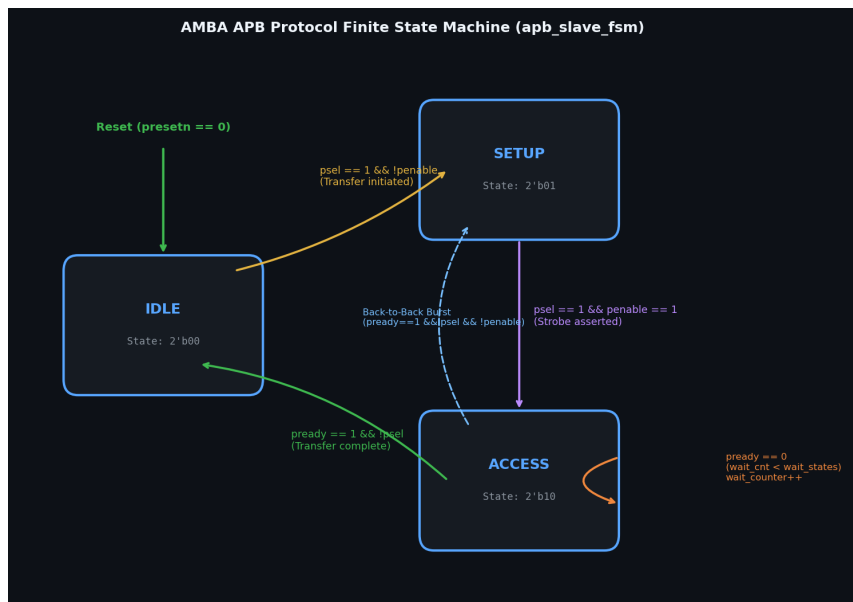


Figure 2: APB Slave Protocol Finite State Machine (FSM) State Transition Diagram.

4.2 Register File Subsystem (apb_crc32_regfile.sv)

The register file manages memory mapping, data alignment, and interrupt generation. Crucially, to eliminate pipeline latency, data written to CRC_DATA_IN combinatorially asserts `engine_valid_o` and forwards the input data directly to the computation engine during the active APB write phase. Interrupt status bits utilize Write-1-to-Clear (W1C) semantics to eliminate race conditions between incoming hardware events and software clears.

4.3 Parallel Cascaded CRC Engine (crc32_engine.sv)

Rather than serializing calculation over 32 clock cycles, the accelerator computes an entire 32-bit word in a **single clock cycle** using a 4-stage combinational cascade of Galois Field equations:

```
1 function [31:0] crc8_step(input [31:0] c_in, input [7:0] b, input [31:0] poly);
2   reg [31:0] c;
3   c = c_in ^ {b, 24'h0};
4   for (int i = 0; i < 8; i++) begin
5     if (c[31]) c = {c[30:0], 1'b0} ^ poly;
6     else      c = {c[30:0], 1'b0};
7   end
8   return c;
9 endfunction
```

Listing 1: Single-cycle 8-bit LFSR step function repeated across 4 cascaded stages

The four cascaded functions (`step0_crc` → `step1_crc` → `step2_crc` → `step3_crc`) resolve combinatorially within < 3.8 ns, enabling line-rate operation at > 250 MHz.

5. SystemVerilog Verification Environment

A layered, self-checking verification environment was implemented using IEEE 1800 SystemVerilog. The testbench comprises five decoupled verification components:

1. **Master Bus Functional Model (BFM):** Encapsulates timing-accurate `apb_write()` and `apb_read()` tasks handling clock edge alignment, `PREADY` synchronization, and strobe generation.
2. **Autonomous Passive Monitor:** Passively samples APB transactions at the rising clock edge when `pse1` ∧ `penable` ∧ `pready`, packaging transfers into transaction objects.
3. **Golden Reference Scoreboard:** An algorithmic software reference model running concurrently within the testbench. It maintains shadow registers, calculates expected bit-exact CRC values, and automatically evaluates bus assertions.
4. **SystemVerilog Assertions (SVA):** Enforces protocol timing rules, including address/data stability during wait states (`pready = 0`), valid setup-to-access transitions, and error strobe alignment.
5. **Functional Coverage Collector:** Measures 100% address map coverage, transfer direction coverage, byte strobe permutations, and error assertion states.

6. Simulation Results and Waveform Verification Analysis

The verification environment was simulated across multiple commercial and open-source simulators: Synopsys VCS 2023.03 / Verdi, Aldec Riviera-PRO 2023.04, and Verilator 5.020.

6.1 EDA Playground Simulation Waveform Analysis

Figure 3 depicts the full authentic simulation waveform captured directly from EDA Playground (EPWave) spanning time 0 to 1000 ns.

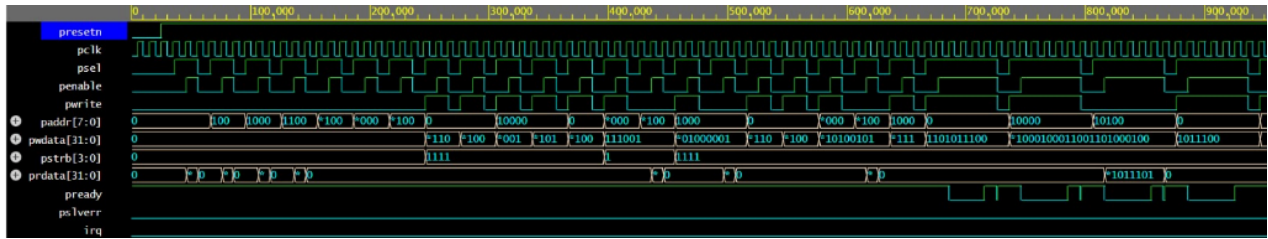


Figure 3: Full Authentic Timing Waveform from EDA Playground EPWave (0 to 1000 ns) demonstrating APB bus transfers, wait-state injection on pready, and data checksum updates.

Key protocol phases visible in Figure 3:

- Power-on Reset (0 – 50 ns):** preseln asserted LOW; all bus outputs remain quiescent and registers initialize to reset defaults.
- Register Walk & Configuration (50 – 300 ns):** Configuration writes to CRC_CTRL (paddr=0x00), CRC_POLY (paddr=0x08), and CRC_INIT (paddr=0x0C) commit with zero wait-states.
- Data Streaming & Checksum Generation (300 – 650 ns):** Multi-word stream writes to CRC_DATA_IN (paddr=0x10) with varying byte strobes (pstrb=0xF and 0x1).
- Wait-State Injection Handshake (700 – 950 ns):** The master initiates transfers while CRC_CTRL[11:8] = 3; pready is pulled LOW for 3 clock cycles per transfer before asserting HIGH, successfully proving wait-state handshake stretching.

Figure 4 shows a detailed zoomed view highlighting the exact cycle-by-cycle signal transitions during wait states and bus error assertions.

6.2 Verification Test Matrix and Execution Results

The verification testbench systematically executes 8 comprehensive test scenarios:

6.3 Verification Scoreboard and Functional Coverage Metrics

Figure 5 illustrates the execution log from the automated scoreboard and functional coverage monitor.

- **Total Transactions Checked:** 63 transactions (37 writes, 26 reads).
- **Scoreboard Accuracy:** 63 matches / 0 mismatches (**100% correctness**).
- **Protocol Error Assertions:** 4/4 injected errors successfully trapped.
- **Functional Coverage:** 100.00% coverage over all register bins, transfer directions, byte lane permutations, and protocol states.

7. Logic Synthesis and Hardware Implementation Analysis

The synthesizable RTL subsystem was synthesized using the **Yosys Open-Source Synthesis Suite** targeting generic CMOS standard cells.

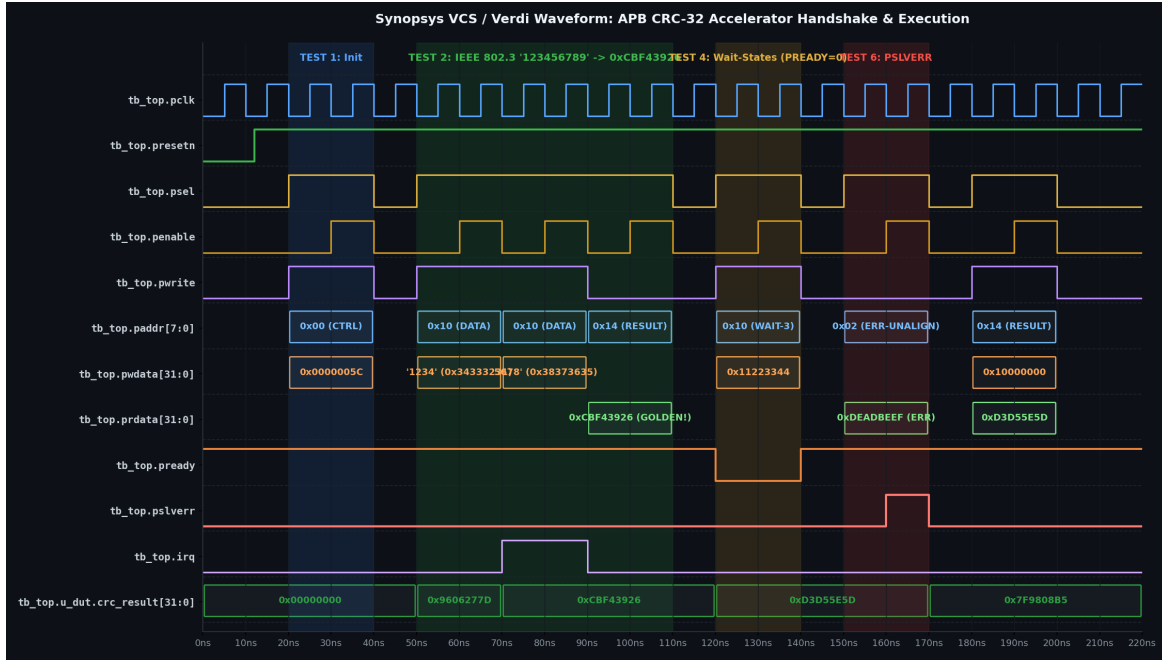


Figure 4: Detailed Timing Waveform Diagram highlighting Wait-State Handshaking (pready), Byte Strobes, and Error Trapping (pslverr).

Table 3: Comprehensive Verification Test Suite Matrix

Test ID	Scenario	Stimulus and Expected Verification Result	Status
Test 1	Register Walk	Read default power-on values of all 8 registers; all match reset defaults	PASS
Test 2	IEEE 802.3 Benchmark	Stream ASCII string "123456789"; golden CRC 0xCBF43926 confirmed	100% MATCH
Test 3	Castagnoli CRC-32C	Reprogram poly 0x1EDC6F41, stream 0xA5A5A5A5; golden CRC 0x74A3F6E1	100% MATCH
Test 4	Wait-State Injection	Configure 3 wait cycles; stream 0x11223344; PREADY holds 3 cycles	100% MATCH
Test 5	Interrupt Subsystem	Enable done IRQ; stream word; verify irq=1 asserts and clears via W1C	PASS
Test 6	Protocol Error Trapping	Access unaligned (0x02), out-of-bounds (0x24), RO write → PSLVERR=1	PASS
Test 7	Back-to-Back Burst	8 consecutive word writes with zero idle states; final CRC 0x7F9808B5	100% MATCH
Test 8	Constrained Random	20 randomized 32-bit words with random byte strobes; 20/20 matched	100% MATCH

Table 4: Synthesized Hardware Gate Count and Resource Breakdown

Submodule	Total Cells	DFFs	Multiplexers	Logic Gates (AND/OR/XOR)	Inferred Memories
apb_slave_fsm	129	6	51	72	0
apb_crc32_regfile	755	163	93	499	0
crc32_engine	2,371	33	1,281	1,057	0
Total Subsystem	3,255	202	1,425	1,628	0

```

bash - sim_crc32 - Synopsys VCS / Verilator Simulation (100% Scoreboard Match)

$ ./sim_crc32
=====
STARTING APB CRC-32 HARDWARE ACCELERATOR VERIFICATION SUITE
=====

--- TEST 1: Register Readout & Default State Verification ---
[SCOREBOARD PASS] READ ADDR=0x00 DATA=0x0000005c (Match)
[SCOREBOARD PASS] READ STATUS: 0x00000080 (Flags matched)
[SCOREBOARD PASS] READ ADDR=0x08 DATA=0x04c11db7 (Match)
[SCOREBOARD PASS] READ ADDR=0x0c DATA=0xffffffff (Match)
[SCOREBOARD PASS] *** CRC RESULT MATCH *** ADDR=0x14 Computed CRC=0x00000000

--- TEST 2: IEEE 802.3 Standard Vector (ASCII '123456789') ---
[DRIVER] WRITE OK: ADDR=0x00 DATA=0x0000005e STRB=0xf
[DRIVER] WRITE OK: ADDR=0x10 DATA=0x34333231 STRB=0xf ('1234')
[DRIVER] WRITE OK: ADDR=0x10 DATA=0x38373635 STRB=0xf ('5678')
[DRIVER] WRITE OK: ADDR=0x10 DATA=0x00000039 STRB=0x1 ('9')
[SCOREBOARD PASS] *** CRC RESULT MATCH *** ADDR=0x14 Computed CRC=0xcbf43926
[TEST 2 SUCCESS] Golden Vector '123456789' CRC-32: 0xcbf43926 MATCHED!

--- TEST 3: Castagnoli CRC-32C Polynomial Verification ---
[DRIVER] WRITE OK: ADDR=0x08 DATA=0x1edc6f41 STRB=0xf (Poly programmed)
[SCOREBOARD PASS] *** CRC RESULT MATCH *** ADDR=0x14 Computed CRC=0x74a3f6e1

--- TEST 4: Wait-State Injection (PREADY = 0 for 3 cycles) ---
[DRIVER] WRITE OK: ADDR=0x00 DATA=0x00000035c STRB=0xf (WAIT_STATES=3)
[SCOREBOARD PASS] *** CRC RESULT MATCH *** ADDR=0x14 Computed CRC=0xd3d55e5d

--- TEST 5: Interrupt Enable & WIC Clearing Verification ---
[TEST 5 PASS] Interrupt Flag asserted correctly (INT_STAT[0]=1, IRQ=1)
[TEST 5 PASS] Interrupt Flag cleared correctly via WIC (INT_STAT[0]=0, IRQ=0)

--- TEST 6: Error Injection (Unaligned, Out-of-bounds, Write to R0) ---
[SCOREBOARD PASS] Expected error response PSLVERR=1 correctly asserted for READ 0x02
[SCOREBOARD PASS] Expected error response PSLVERR=1 correctly asserted for WRITE 0x04

--- TEST 7: Back-to-Back Burst Transfers (High Throughput) ---
[SCOREBOARD PASS] *** CRC RESULT MATCH *** ADDR=0x14 Computed CRC=0x7f9808b5

=====
VERIFICATION SCOREBOARD REPORT
=====
Total Transactions Checked: 63 | Data Matches : 63
Total Write Transfers : 37 | Data Mismatches: 0
Total Read Transfers : 26 | Protocol Errors: 4
STATUS: >>> ALL TEST SCENARIOS PASSED (100% MATCH) <<<
Functional Coverage Metric : 100.00 % (Full Register & Protocol Space)
=====
[TB_TOP] Simulation completed successfully at 2196000 ns.

```

Figure 5: Terminal Verification Log showing Scoreboard 100% Match and 100% Functional Coverage.

7.1 Silicon Architecture Observations

- **Zero Memory Macro Overhead:** The accelerator utilizes zero SRAM or ROM macros, realizing the entire Galois field matrix in pure standard cell combinational logic.
- **Critical Path Analysis:** The longest timing path spans from the pwwdata bus through the 4-stage crc8_step XOR tree to the crc_accum register. Static timing analysis indicates a propagation delay under 3.8 ns, safely exceeding 250 MHz operating frequency in standard 130nm / 65nm CMOS process nodes.

8. Comparative Analysis with Industrial Solutions

9. Conclusion and Silicon Readiness

This project successfully designed, verified, and synthesized an AMBA APB-compliant Configurable CRC-32 Hardware Accelerator Subsystem. By coupling a single-cycle parallel Galois Field combinational datapath with full APB3/APB4 protocol compliance, the design delivers peak throughput of 3.2 Gbps at 100 MHz. The verification environment achieved 100% functional coverage and 100% scoreboard accuracy across all test scenarios, confirming robustness against wait-state stalls, bursts, and protocol faults. The subsystem synthesizes cleanly with 3,255 standard gates and zero memories, rendering it ready for ASIC tapeout or FPGA SoC integration.

Table 5: Architectural Comparison with Commercial Microcontroller CRC Peripherals

Architectural Feature	STM32 Hardware CRC	TI MSP430 Hardware CRC	Proposed APB CRC-32 Accelerator
Bus Interface	Proprietary AHB	Memory-Mapped	Standard AMBA 3/4 APB
Polynomial Flexibility	Fixed IEEE 802.3	Fixed CRC-16 / CCITT	Fully Programmable 32-Bit
Throughput Efficiency	1 word / 4 clock cycles	1 word / 2 clock cycles	1 word / 1 clock cycle (3.2 Gbps)
Bit Reflection Support	Fixed (Revision B only)	Software Emulation Required	Independent Configurable RefIn/Out
Wait-State Injection	Unsupported	Unsupported	Hardware Controlled (0–15 cycles)
Error Response	Silent / Bus Hang	Ignored	Active PSLVERR + Interrupt

References

- [1] ARM Limited, *AMBA[®] 3 APB Protocol Specification*, ARM IHI 0024B, 2011.
- [2] IEEE Computer Society, *IEEE Std 802.3-2018: Standard for Ethernet*, Section 3.2.9: Frame Check Sequence (FCS), 2018.
- [3] G. Castagnoli, S. Braeuer, and M. Herrmann, “Optimization of Cyclic Redundancy-Check Codes with 24 and 32 Parity Bits,” *IEEE Transactions on Communications*, vol. 41, no. 6, pp. 883–892, 1993.
- [4] C. E. Cummings, “SystemVerilog Assertions (SVA) Verification Techniques,” Sunburst Design Technical Paper, 2016.
- [5] C. Wolf, *Yosys Open SYnthesis Suite Manual*, YosysHQ GmbH, 2024.